Material de Estudo
Estruturas de Dados I

Prof. Tiago Pessoa Ferreira de Lima

Conteúdo

1 Revisão Rápida: Busca e Ordenação 1

2 Questão 1: Busca Binária Iterativa 2

3 Questão 2: Bubble Sort Otimizado 2

4 Questão 3: Selection Sort 3

5 Questão 4: Insertion Sort 4

6 Dicas Gerais para a Prova 4

7 Exemplos de Testes 5

---

1 Revisão Rápida: Busca e Ordenação

Em Estruturas de Dados I, trabalhamos com coleções lineares (como listas e arrays) e focamos em duas das operações mais fundamentais da computação: encontrar elementos (Busca) e organizá-los (Ordenação).

Algoritmos de Busca:

- **Busca Sequencial (Linear):** Percorre a lista elemento por elemento. Funciona em listas não ordenadas.
- **Busca Binária:** Divide a lista pela metade a cada passo. **Requisito:** A lista deve estar obrigatoriamente ordenada.

Algoritmos de Ordenação $O(n^2)$:

- **Bubble Sort:** "Flutua"o maior elemento para o final fazendo trocas adjacentes.
- **Selection Sort:** Seleciona o menor elemento da porção não ordenada e o coloca na posição correta.
- **Insertion Sort:** Constrói a lista ordenada elemento por elemento, inserindo cada novo item em sua posição correta na sublista já avaliada.

2 Questão 1: Busca Binária Iterativa

Enunciado: Implemente a função `busca_binaria(lista, alvo)` que recebe uma lista de números inteiros ordenados em ordem crescente e um valor `alvo`. A função deve retornar o índice do alvo na lista. Se o alvo não estiver na lista, retorne `-1`.

Exemplo: Para `lista = [2, 5, 7, 10, 15, 20]` e `alvo = 10`, a função deve retornar `3`. Para o `alvo = 8`, a função deve retornar `-1`.

Estratégia:

- Utilizar dois ponteiros, `inicio` e `fim`, delimitando o espaço de busca.
- Calcular o `meio`. Se o elemento do meio for o alvo, retornar seu índice.
- Se o alvo for menor que o elemento do meio, ajustar o `fim` para `meio - 1`.
- Se o alvo for maior, ajustar o `inicio` para `meio + 1`.

Implementação:

```python
def busca_binaria(lista, alvo):
    inicio = 0
    fim = len(lista) - 1

    while inicio <= fim:
        meio = (inicio + fim) // 2

        if lista[meio] == alvo:
            return meio
        elif lista[meio] < alvo:
            inicio = meio + 1
        else:
            fim = meio - 1

    return -1
```

Listing 1: Método de Busca Binária

Complexidade:

- Tempo: $O(\log n)$ no pior caso.
- Espaço: $O(1)$ pois utiliza apenas variáveis auxiliares.

3 Questão 2: Bubble Sort Otimizado

Enunciado: Implemente a função `bubble_sort(lista)` que ordena uma lista in-place. A implementação deve ser otimizada para interromper a execução caso a lista já esteja ordenada antes de terminar todas as passagens.

Estratégia:

- Percorrer a lista comparando elementos adjacentes e trocando-os se estiverem na ordem errada.
- Utilizar uma variável booleana `trocou` no início de cada passagem. Se ao final da passagem ela continuar `False`, significa que a lista já está ordenada e podemos parar o laço.

Implementação:

```python
def bubble_sort(lista):
    n = len(lista)
    for i in range(n):
        trocou = False
        # Otimizacao: ignorar os ultimos i elementos ja ordenados
        for j in range(0, n - i - 1):
            if lista[j] > lista[j + 1]:
                # Troca os elementos
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
                trocou = True

        # Se nao houve troca nesta passada, a lista esta ordenada
        if not trocou:
            break
```

Listing 2: Método Bubble Sort Otimizado

Complexidade:

- Tempo: $O(n^2)$ no pior caso (lista invertida). $O(n)$ no melhor caso (lista já ordenada, graças à otimização).
- Espaço: $O(1)$.

4 Questão 3: Selection Sort

Enunciado: Implemente a função `selection_sort(lista)` que ordena uma lista in-place buscando o menor elemento e movendo-o para o início da porção não ordenada da lista.

Estratégia:

- Dividir logicamente a lista em duas partes: ordenada (à esquerda) e não ordenada (à direita).
- Iterar pela lista e assumir que o primeiro elemento da parte não ordenada é o menor.
- Buscar o verdadeiro menor elemento no resto da lista e trocar com o elemento atual.

Implementação:

```python
def selection_sort(lista):
    n = len(lista)
    for i in range(n - 1):
        # Assume que o menor elemento e o primeiro da porcao nao ordenada
        indice_minimo = i

        # Busca o menor elemento no resto da lista
        for j in range(i + 1, n):
            if lista[j] < lista[indice_minimo]:
                indice_minimo = j

        # Troca o menor elemento encontrado com o elemento da posicao i
        if indice_minimo != i:
            lista[i], lista[indice_minimo] = lista[indice_minimo], lista[i]
```

Listing 3: Método Selection Sort

Complexidade:

- Tempo: $O(n^2)$ em todos os casos (melhor, médio e pior), pois ele sempre faz a varredura completa buscando o menor.
- Espaço: $O(1)$.

5 Questão 4: Insertion Sort

Enunciado: Implemente a função `insertion_sort(lista)` que ordena uma lista in-place inserindo cada elemento na sua posição correta dentro de uma sublista já ordenada.

Estratégia:

- Considerar o primeiro elemento (índice 0) como uma sublista já ordenada.
- Pegar o próximo elemento (chave) e compará-lo com os elementos da sublista ordenada, movendo os elementos maiores uma posição para a direita.
- Inserir a chave na posição correta e repetir para todos os elementos.

Implementação:

```python
def insertion_sort(lista):
    n = len(lista)
    for i in range(1, n):
        chave = lista[i]
        j = i - 1

        # Move os elementos maiores que a chave uma posicao para a direita
        while j >= 0 and lista[j] > chave:
            lista[j + 1] = lista[j]
            j -= 1

        # Insere a chave na posicao correta
        lista[j + 1] = chave
```

Listing 4: Método Insertion Sort

Complexidade:

- Tempo: $O(n^2)$ no pior caso. $O(n)$ no melhor caso (lista já quase ou totalmente ordenada).
- Espaço: $O(1)$.

6 Dicas Gerais para a Prova

1. **Cuidado com os índices:** Em Python, listas são indexadas a partir do zero. Muita atenção com os limites do `while` e do `range()` para evitar `IndexError`.
2. **Teste de Mesa no Papel:** Trace os algoritmos à mão com vetores pequenos (ex: `[5, 2, 4, 1]`). Isso é essencial para internalizar como os ponteiros se movem.
3. **In-place vs Nova Lista:** Observe se a questão pede para alterar a lista original ("in-place", comportamento padrão destes algoritmos de ordenação em Python) ou se pede para retornar uma lista nova.
4. **Entenda a característica de cada algoritmo:** Saiba diferenciar qual método faz trocas em massa adjacentes (Bubble) e qual empurra os elementos arrastando para a direita (Insertion).

7 Exemplos de Testes

Você pode criar um pequeno bloco de código para verificar suas implementações e observar a mutabilidade das listas no Python:

```python
# Teste da Busca Binaria
lista_ordenada = [1, 3, 5, 7, 9, 11, 15]
print("Busca Binaria (alvo 7):", busca_binaria(lista_ordenada, 7))  # 3
print("Busca Binaria (alvo 8):", busca_binaria(lista_ordenada, 8))  # -1

# Testes de Ordenacao
lista1 = [64, 34, 25, 12, 22, 11, 90]
bubble_sort(lista1)
print("Bubble Sort:", lista1)

lista2 = [64, 25, 12, 22, 11]
selection_sort(lista2)
print("Selection Sort:", lista2)

lista3 = [12, 11, 13, 5, 6]
insertion_sort(lista3)
print("Insertion Sort:", lista3)
```

Listing 5: Exemplos de teste

Bons estudos! Lembre-se de praticar a escrita dos métodos à mão também, pois na prova você não terá um computador para testar. Treine a lógica no papel.